

问题一代码架构与模型详解

模块职责 数据流转 模型原理 运行机制

本项目的核心任务，是把附件 1 的 100 条原始英文视频及配套转写，转换为可逐词回查的文本、语音和视觉特征。理解代码时，最重要的是区分三个层次：预训练工具负责产生基础表示；对齐与聚合算法负责建立时间关系；运行、核验和存储模块负责保证结果能够读取、检查和复现。整个系统是离线批处理程序，没有网页服务，也没有情感预测模型的训练流程。

1 系统定位与总体架构

1.1 输入和输出是什么

输入来自附件 1：原始 MP4 视频和 label-100.xlsx。表格中的 video_id 与 clip_id 共同确定视频文件，text 提供已知英文转写，label 及 annotation 提供情感标注。程序利用转写提取文本表示并约束语音对齐，标签只登记到元数据及汇总中，不作为训练目标。附件 2 的现成特征未用于替代本问的自主提取。

输出的主体是一组数值矩阵，而不是“积极、消极、中性”的预测。如果一条视频对应 L 个词，该样本输出 L 行文本特征、L 行语音特征和 L 行视觉特征，列数分别为 768、50 和 110。相同行号表示同一个原词对应的时间区间。某模态没有有效观测时，行位置仍保留，通过掩码说明不可用原因或状态。

1.2 为什么需要多个模块

视频读取、模型推理、时间对齐和报告生成属于不同职责。如果全部混在一个循环中，后续很难判断是源数据有问题、模型无输出，还是保存与读取规则不一致。现有代码把数值对齐算法拆到 alignment.py，把读取与补齐拆到 read_features.py，并把全量核验和图表单独放在 summarize.py 中；pipeline.py 仍承担大部分提取与调度工作。

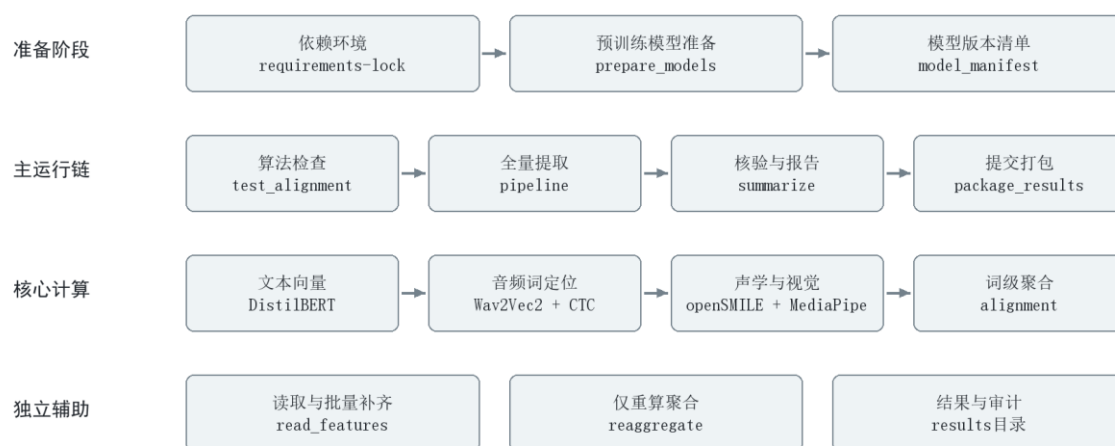


图 1 项目各阶段与核心计算模块的关系

阅读顺序可从运行入口进入主程序，再按特征支路理解模型，最后检查结果契约和异常策略。以下各节按这一顺序展开，保留真实文件名与函数名作为定位线索，不展示源代码。

2 文件职责与实际调用关系

表 1 项目文件在系统中的分工

文件	负责的工作	调用或使用时机
run.ps1	顺序调度检查、提取、汇总和打包	已有依赖与模型后的主入口
pipeline.py	读取清单、加载提取器、逐样本处理与保存	主提取程序
alignment.py	分词、CTC 动态规划、区间聚合和补集	由提取和重聚合模块导入
prepare_models.py	下载模型、固定修订号并登记文件哈希	首次准备或缓存维护
config.json	记录采样率、阈值、分块及线程等设置	提取器初始化时读取
summarize.py	独立复算、汇总统计、案例图与报告	提取后检查结果
read_features.py	读取单样本并形成批量输入	后续分析或建模读取
test_alignment.py	执行 6 项确定性算法检查	主运行链的第一步
reaggregate.py	从保存的帧级数据重算词级统计	仅聚合精度调整时单独使用
package_results.py	检查完成状态、压缩结果及检查包大小	汇总校验通过之后
requirements-lock.txt	记录实际依赖版本	环境建立与复现
README.md	解释运行顺序、字段、掩码与局限	人工阅读入口

2.1 主入口不是安装程序

run.ps1 查找项目根目录中的虚拟环境解释器，先执行算法检查，再运行带断点续跑选项的提取程序，接着进行汇总验证，最后打包。每一步检查退出状态；前一步失败时终止，不继续执行后续阶段。因此，测试失败不会进入提取，提取出现失败样本也不会正常进入打包。

该入口不负责创建虚拟环境，不自动安装依赖，也不自动下载模型。prepare_models.py 属于前置准备；模型目录或依赖缺失时，应先完成准备步骤。将“准备阶段”和“正常运行阶段”分开，能够让正式处理只从本地读取模型，不在样本处理过程中访问模型下载服务。

2.2 两条辅助路径

read_features.py 独立于提取链，用于消费已有输出；运行它不需要重新解码视频或启动深度模型。reaggregate.py 则复用已保存的帧特征、词时间区间与有效掩码，只更新语音和视觉的词级统计量。两者都利用了“原始帧级特征与最终词级特征同时保存”的设计。

现有程序没有通用任务队列、分布式调度或数据库。模型下载最多并发 3 个任务，但 100 条视频由主程序依次处理；PyTorch 的 4 线程配置属于单进程内部计算资源设置，不等于 4 条视频同时提取。

3 模型准备 配置和运行资源

3.1 预训练模型如何进入项目

prepare_models.py 准备两个 Hugging Face 模型目录和一个 MediaPipe 任务文件。文本目录保存 DistilBERT 权重、配置、词表与分词器；对齐目录保存 Wav2Vec2 权重、处理器与 CTC 词表；视觉使用 face_landmarker.task。它们是公开预训练模型资源，不是由附件 1 训练出来的模型。

当模型清单尚未记录某仓库时，准备程序解析当前仓库修订号，并将后续下载地址固定到该修订。下载先写入临时文件，完成后替换为正式文件名；失败最多尝试 4 次。首次完整下载后计算文件 SHA256 并登记，后续准备时与此前摘要核对。若完全删除模型清单，则重新准备可能解析到其他修订，不能再认为与原实验完全一致。

3.2 参数集中管理的范围

表 2 影响处理行为的配置

配置项	当前值	实际作用
音频采样率	16000 Hz	公共音频数组与声学模型输入
目标视频采样频率	10 Hz	决定选取哪些视频帧
视频最大宽度	640 像素	限制检测输入宽度并保持比例
人脸检测与存在阈值	均为 0.5	筛选视觉检测结果
文本块大小	100 个原词	长文本的分块编码门限
对齐复核阈值	0.1	产生低分词复核标记
PyTorch 线程数	4	限制 CPU 计算线程
随机种子	20260923	固定相关随机状态
输出数值类型	float32	保存的特征精度

配置并非覆盖所有实现细节。数字读法映射、最多检测 3 张人脸、95% 的声学观测门限、视觉支持区间截断规则等仍写在提取代码中。配置中的模型名称用于描述选择，但实际加载入口指向本地 models/text 和 models/alignment 目录；仅修改名称字符串，并不会自动下载或替换模型。

3.3 提取器如何复用资源

Extractor 在逐样本循环之前创建一次，加载文本编码器、声学 CTC 模型、openSMILE 实例和人脸检测任务，随后由全部样本复用。这样避免每处理一条视频都重新加载权重。两个神经网络设为评估模式，前向计算在推理模式中进行，没有优化器、梯度回传或参数更新；当前代码也没有将模型转移到 GPU 的步骤。

Windows 下，openSMILE 配置树被复制到临时目录以适配路径限制，音频通过内存数组传入。常规结束时关闭 MediaPipe 任务并清理临时副本。异常终止时的资源回收仍依赖运行路径，应与正常收尾机制区分。

4 主流水线如何处理一条样本

4.1 建立可靠的样本清单

`read_manifest` 递归寻找唯一的 `label-100.xlsx`，读取每行的样本编号、文本和标签。`clip_id` 若被表格读成带“.0”的数值形式，会先转为文件名需要的整数文本。视频路径由表格所在目录、`video_id` 子目录和 `clip_id.mp4` 组合得到。程序要求 100 条唯一记录与视频集合完全一致，并检查标签范围及极性对应关系；这些检查在模型提取前完成。

4.2 公共时间轴先于特征提取

`media_audio` 先获取容器起始时间和视频基本信息，再解码音频并转换为 16 kHz 单声道。每个重采样片段依据自身真实时间戳写入统一数组，另有布尔数组记录哪些采样点确实来自解码数据。数组中的零既可能是真实零振幅，也可能是未观测占位，必须结合观测数组解释。

视频端在后续独立解码时同样减去容器原点。两条支路因此使用统一的相对秒数，而不是将词号、声学帧号和视频帧号直接视为同一种时间索引。原视频的音画质量仍限制最终对应关系，公共坐标本身不保证源素材没有同步误差。

4.3 `process_one` 的执行顺序

表 3 一条样本在主流程中的状态变化

步骤	形成的中间对象	设计目的
读取媒体并分词	音频数组、观测数组、词序列、字符偏移	明确时间原点与词索引
计算文本表示	每词 768 维向量及子词映射	保留上下文语义表示
进行强制对齐	词起止时间、分数与对齐掩码	建立词与秒数的对应
提取两类帧特征	声学 25 维、视觉 55 维及支持区间	保留模态自身时间尺度
按词区间聚合	每词 50 维语音和 110 维视觉向量	统一三模态序列位置
登记质量状态	低分、无人脸、多人脸等提示	区分完成状态与观测质量
检查并保存	NPZ 数组文件与 JSON 元数据	形成可读取、可追溯输出

4.4 完成不等于可靠

只要能够生成完整字段，样本就可能保存为“需要复核”而不是失败。比如人脸缺失会导致部分视觉词向量置零，但文本与语音仍可保存；低分路径保留区间并标记。对齐函数抛出 `ValueError` 时，`process_one` 保留原词位置，将区间、分数和对齐掩码设为无效，再继续提取原始模态特征。其他无法完成样本的异常由外层循环记录为 `failed`。

主程序会继续尝试后续样本，最后根据失败计数返回失败状态。这使单个视频异常不必立即丢失整批处理机会，但仍能阻止一次含失败样本的运行被当作完整成功。

5 文本模型 DistilBERT 的作用与结构

5.1 模型在项目中回答什么问题

DistilBERT 负责把文本转成连续数值表示。它根据词在上下文中的用法生成语义向量，使同一单词出现在不同句子时能够产生不同表示。项目选择 `distilbert/distilbert-base-uncased`，使用基础编码器的最后隐层输出，并没有加载或训练面向本题的情感分类头。故 768 个数表示语义空间中的坐标，不是 768 种情绪，也不是一组可直接相加解释的情感分数。

5.2 从原词到子词再回到原词

原始转写先由正则规则拆成词，同时保留字符偏移。英文缩写中的撇号、带千分位的数字和序数词被专门处理，连字符两侧可以拆成不同词；标点仍在原文中保存，但不作为独立特征位置输入编码器。模型分词器随后将每个词转换成一个或多个子词，子词能够帮助表示低频词及词形变化。

子词序列经过嵌入与 Transformer 编码，得到每个子词的 768 维向量。程序利用 `word_ids` 找到子词归属，对同一原词的最后层向量取平均。这样，模型内部可以使用子词，而输出仍与后面的原词时间区间一一对应。元数据保存子词位置、编号及所属文本块，允许检查合并过程。

表 4 本地 DistilBERT 配置中的关键结构

结构参数	记录值	理解方式
Transformer 层数	6	多层上下文编码
每层注意力头数	12	从多个表示子空间建立词间联系
隐藏向量维数	768	每个子词最终输出的长度
前馈网络中间维数	3072	层内非线性变换容量
词表大小	30522	分词器可映射的词元编号集合
最大位置数	512	单次编码的模型位置上限
激活函数	GELU	前馈变换中的非线性操作

5.3 分块策略和使用边界

代码每 100 个原词形成一块，但模型上限按子词位置计算，两者不能混同。即使原词数未到上限，若拆出的子词过多，程序也会显式报错，而非静默截断。当前最长样本 65 词，不需要跨多个词块；未来长文本若被分块，跨块上下文不会自动共享。

`uncased` 意味着模型不区分英文大小写；实际输入还经过原词分词，因此标点和大小写线索未被完整保留。模型预训练提供的是通用语言知识，当前没有用附件 1 情感标签校准其输出。对具体维度的情绪含义、分类效果或解释能力，都不能仅凭采用了 DistilBERT 就作结论。

6 Wav2Vec2 模型与词时间定位

6.1 它是对齐工具而非语音情感特征来源

项目使用 facebook/wav2vec2-base-960h，将 16 kHz 音频转为逐时间步的字符概率。声学模型提供“此时像哪个字符”的证据，再由强制对齐算法判断给定转写应落在何处。最终 50 维语音特征来自 openSMILE，不是从 Wav2Vec2 隐层直接截取；区分这两个音频支路是理解架构的关键。

6.2 模型内部的主要处理层次

音频首先经过 7 层一维卷积，将高频采样点压缩为较低时间分辨率的声学表示；随后进入 12 层 Transformer，在较长上下文中建模声音关系；CTC 输出头将每个时间步的表示映射到 32 个词表符号的得分，包含英文字母、词分隔符及特殊符号。程序对得分计算对数概率，交给动态规划使用。

表 5 本地 Wav2Vec2 配置与项目中的含义

配置	记录值	说明
卷积层数与通道数	7 层；各 512 通道	逐层提取并降低时间分辨率
卷积核大小	10、3、3、3、3、2、2	决定局部感受野
卷积步幅	5、2、2、2、2、2、2	总步幅 320 个音频采样点
Transformer 结构	12 层；12 个注意力头	声学上下文建模
隐藏维数与前馈维数	768；3072	内部表示规模
CTC 词表大小	32	字符与特殊符号输出
时间步长	约 20 毫秒	16 kHz 下 320 个采样点的间隔

6.3 已知文本如何约束声音

项目不是先自由识别一句新文本，再把新文本替换到题目中。它将赛题转写大写化，并按显式映射展开少量数字读法，保留每个字符所属的原词。词间分隔符“|”作为目标符号参与匹配；CTC blank 则是另一个特殊状态，用于处理发射停留及重复字符。两者职责不同。

动态规划寻找能完整发射给定目标序列的最大概率路径。对于连续重复字符，必须经过 blank 分离，不能用一个长发射段同时代表两个相同字母。回溯后先获得字符时间范围，再用每个词首末字符的边界组成词区间，最后依据卷积感受野中心与步长换算成秒。

6.4 分数与诊断文本如何解释

每个字符先在其发射帧上计算几何平均概率，再对词内字符计算几何平均，形成词分数。阈值 0.1 只用于提示复核，不能解释为 90%的错误概率。声学模型的贪心识别文本也会另行保存，用来与原转写比较，但它不覆盖原文。若原文与声音不匹配，算法仍可能找到合法但语义错误的路径；20 毫秒步长也不等于 20 毫秒边界精度。

7 openSMILE 声学特征支路

7.1 工具和预训练神经网络的区别

openSMILE 是声学信号处理与特征提取工具，在本项目中根据 eGeMAPSv02 配置计算低层描述符，不进行神经网络训练，也没有通过情感标签学习一组声学权重。它把音频变成具有声学意义的数值，例如响度、频谱变化、基频与音质相关量。因此，项目中所称的“模型与工具”包含不同类型：DistilBERT 和 Wav2Vec2 是神经网络，openSMILE 主要执行固定配置的信号处理。

7.2 为什么保留帧级描述符

如果直接计算整句统计特征，后续无法把句子内部的声学变化准确分配给某个词。代码选择 LowLevelDescriptors 层级，取得每帧 25 维特征及工具返回的开始、结束时刻。等到词区间确定后，再对相交的声学帧做加权统计，使每个词有各自的语音表示。这里的 25 维帧级输出与整句 88 维统计输出是不同接口。

表 6 声学特征从原始帧到词级表示

特征类别	帧级内容	在理解声音时的作用
能量与频谱	响度、频谱比值、谱斜率、谱通量	描述强弱及频谱变化
倒谱	前 4 阶 MFCC	概括谱包络形状
基频与音质	半音基频、抖动、微颤、谐噪比	描述音高及发声稳定性
谐波差异	H1-H2 及 H1-A3 相关量	反映部分发声特征
共振峰	前三个共振峰的频率、带宽及相对幅度	描述声道共振信息
词级输出	25 维均值与 25 维标准差拼接	同时保留平均状态与变化幅度

7.3 有效声学帧的判断

程序先统计声学帧区间内有多少采样点确实来自音频解码，只有观测比例至少 95%且所有特征值为有限数时才记为有效。未通过的帧用零占位，但在聚合时由掩码排除。这里的有效表示“观测及数值条件合格”，并不意味着该帧必然是清晰语音，也不等于已确认其发音属于当前词。

某些描述符在无声或清辅音附近可能为零，不能仅凭零值判定缺失。尤其应区分音频解码缺口、真实静音以及某个声学量不适用的情况。现有代码的 95%门限负责观测完整度，不包含独立的语音活动检测、降噪或人工音质审核。

7.4 输出尺度与后续使用

不同描述符的单位和数量级不同，代码没有在全量 100 条样本上拟合标准化器。聚合采用原始量纲，均值与标准差仍对应各自声学量。后续若进入学习模型，标准化参数应由相应训练数据估计并固定；不能为方便而把测试样本也用于拟合统计量。当前文档描述的是实际提取接口，不代表已经完成该后续步骤。

8 MediaPipe 视觉支路

8.1 从视频解码到候选人脸

视觉支路按真实视频时间戳进行 10 Hz 目标抽样，并登记原视频中从零开始的帧号。每个保留帧转换为 RGB；若宽度超过 640 像素，则按比例缩小。之后使用 MediaPipe Face Landmarker 任务，读取人脸关键点、表情混合形状系数和面部变换矩阵。

任务采用 IMAGE 模式，逐个采样帧独立检测；不是开启 VIDEO 模式后持续跟踪同一身份。每帧最多检测 3 张脸，并从各脸关键点的包围框中选择面积最大者。若画面切换或两人交替接近镜头，被选目标也可能切换，因此“最大人脸”只能称为候选表达主体，不是已验证的发声人。

8.2 52 维表情系数与 3 维头姿

检测成功后，程序读取 52 个 blendshape 系数，涉及眉眼、面颊、下颌和口部等形状变化。它们表示模型估计的混合形状权重，不是积极或消极情绪的概率，也不能直接改名为 OpenFace 的 FACS 动作单元。某个微笑相关系数上升可以作为动作线索，但不能脱离上下文直接推出情感标签。

面部变换矩阵还包含姿态信息。程序取左上角 3×3 矩阵，通过奇异值分解与行列式校正得到合法旋转，再计算 xyz 顺序的 3 个欧拉角，单位为弧度。52 个表情系数与 3 个角度拼接成 55 维帧级向量；词内均值和标准差再拼接为 110 维。

表 7 视觉支路保留的信息

信息	保存形式	可以检查什么
原始 55 维视觉表示	vision_raw	每个采样时刻的表情与头姿
支持区间及真实时刻	vision_intervals、vision_pts	哪些时间范围由该帧代表
原视频帧号	vision_frame_ids	回查源视频画面
人脸数量	face_counts	是否有多人脸候选
被选人脸包围框	face_boxes	算法选中了画面中的哪一部分
原始帧有效性	vision_raw_mask	检测及数值是否有效
词级有效性	vision_mask	该词是否获得有效视觉贡献

8.3 为什么要给单帧一个时间区间

视频帧只对应一个采样时刻，但词本身持续一段时间。为进行区间聚合，代码以相邻采样时刻的中点划分支持范围，单帧向左右最多延伸 0.075 秒，并限制在片段范围内。这相当于假定该短区间内可以由当前帧近似代表。检测失败的帧不参与统计，较大的解码空隙也不会被无限延伸的支持区间填满。

10 Hz 抽样降低了计算量，但可能漏掉更短暂的表情变化；独立帧检测和最大人脸选择也不保证时间上身份连续。输出掩码能记录可用性，不能恢复未观测的动作。当前代码没有进行表情插值补全、身份跟踪或说话人确认。

9 三模态如何汇合为同一序列

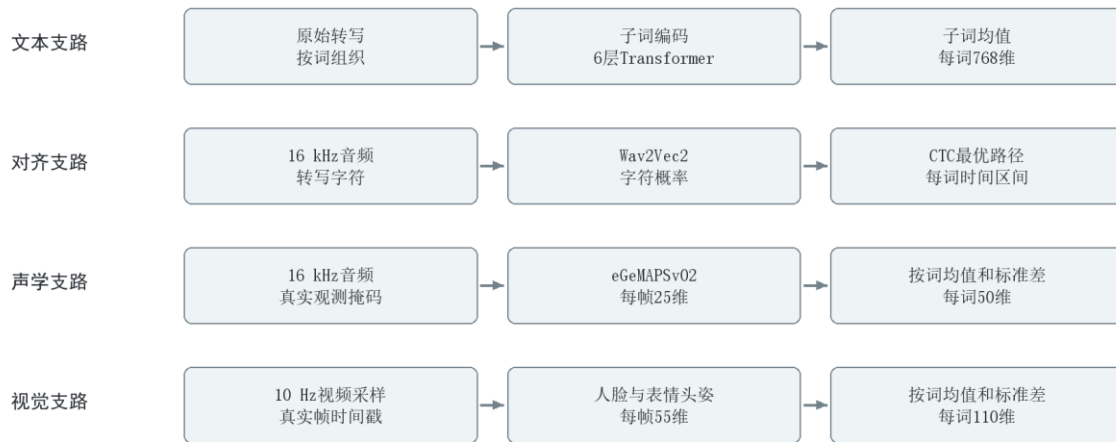


图 2 四条计算支路与最终三模态表示的关系

9.1 时间对齐与特征融合不是同一件事

Wav2Vec2 和 CTC 提供词时间区间；文本向量天然以词为单位；声学和视觉输出仍以各自帧为单位。alignment.py 中的 pool_intervals 负责把后两者转换到词级。它不会学习模态权重，不会将文本、语音、视觉拼接为一个情感分类器，也没有跨模态注意力网络。当前完成的是输入组织与对齐，而非最终情感融合决策。

9.2 时间重叠权重如何工作

对一个词，算法逐一检查源帧的时间支持区间，计算二者交集持续多久。重叠越长、且有效掩码为真，该帧在均值和标准差中贡献越大；不相交或无效的帧贡献为零。源帧并不需要中心点落在词区间内部，只要支持区间存在有效重叠，就可能参与统计。

例如目标区间为 0.5 至 2.0 秒，两个有效源区间分别为 0 至 1 秒和 1 至 2 秒，对应数值 2 和 4。两个权重是 0.5 和 1.0，所以结果偏向第二帧，加权均值约为 3.333，而不是直接平均得到 3。该规则同时用于声学和视觉，保证两种模态的时间聚合遵循相同定义。

9.3 掩码之间的关系

表 8 常见布尔标记的真实含义

标记	True 表示	不代表什么
text_mask	真实保留的原词位置	该词有可靠声学时间
audio_mask / vision_mask	至少存在有效源帧贡献	整个词区间完整被观测
alignment_mask	找到合法受约束路径	人工确认语义与时间正确
alignment_review_mask	对齐低分或失败，需要复核	已证明该词一定错误
padding_mask	批量补齐产生的人为位置	原始模态缺失

当没有有效源帧时保留该词位置、输出零向量并关闭相应模态掩码；当只是对齐低分时，路径和模态特征仍保留。因此，使用者若需要可靠时间证据，应同时检查模态可用性、对齐有效性和复核标记，不能只读取特征矩阵。

10 结果目录 数据契约与读取方式

10.1 数值文件与解释信息分开保存

NPZ 适合保存压缩数值数组，JSON 适合保存样本来源、文本与处理说明。两者以同一个样本编号命名，并由 metadata 中的 feature_file 建立关联。读取时禁止 pickle 对象，获得的主要是数组和标准 JSON 值。这个设计使结果分析可以不依赖原始模型对象，从而将“生产特征”和“消费特征”分开。

表 9 结果目录中的主要材料

材料	阅读价值
features 与 metadata	100 条样本的数值特征及逐条来源说明
manifest.json	核对预期样本编号、文本和标签
feature_names.json	确定每个特征列的名称与顺序
sample_summary.csv	每样本一行，检查规模、覆盖率与告警
modality_summary.csv	每样本每模态一行，共 300 行
word_alignment.csv	检查全部词时间、分数与模态有效性
alignment_diagnostics.csv	对照原转写和声学模型识别文本
manual_review_template.csv	记录人工边界和回看结论，空白不是零误差
validation.json 与 run_environment.json	检查自动核验结果、版本与参数
processing_log.jsonl	按处理事件追加的状态日志

10.2 证据如何从词回到源视频

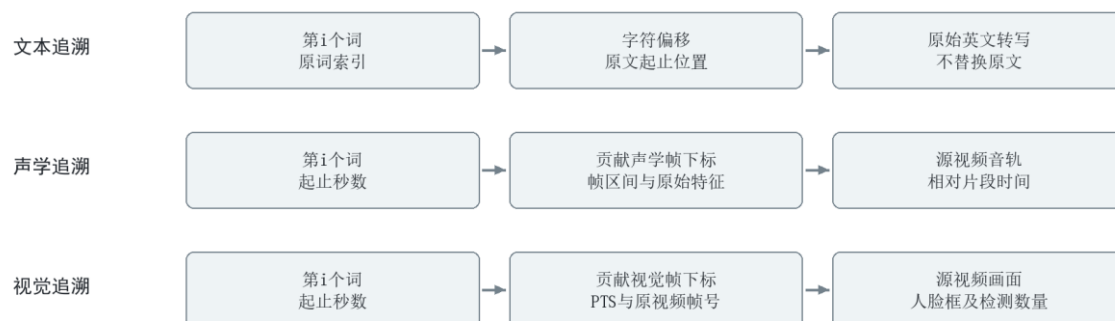


图 3 同一词位置的文本 声学与视觉追溯路径

10.3 变长序列如何进入批量计算

read_sample 按样本编号读取一对文件，不改变序列长度。collate 再统计当前批次各样本长度，按批内最大值建立统一数组：真实词位置复制原始内容，补齐位置的模态值为零，padding_mask 为 True，词时间为-1，模态有效掩码为 False。磁盘原文件不做补齐，也不截成 50 词。

valid_length 是保留的词数，不是“所有模态同时有效”的词数。以当前数据中 65 词样本为例，它可以完整进入批次；同批较短样本才补到相同长度。后续模型应使用补齐掩码排除人为位置，同时独立处理真实词位置上的模态缺失。

11 质量核验 复算与运行状态

11.1 两种检查各自证明什么

test_alignment.py 用 6 个小型、结果可确定的案例检查关键算法：重复字符必须经过 blank；过短输入不应产生合法重复字符路径；加权聚合应排除无效帧；字符偏移能回读原文；数字分词保留千分位及序数形式；批量补齐保留超过 50 词的样本。这些检查可较快发现边界逻辑错误，但不验证真实视频的识别质量。

summarize.py 针对实际 100 条输出检查形状、有效长度、时间顺序、有限数值、原文与标签一致性和输出文件哈希，并用单独编写的逐区间参考实现从帧特征重算词级均值和标准差。它还检查模态掩码与贡献帧下标，使数值结果和来源记录同时受约束。数值比较使用相对与绝对容差均为 2×10^{-5} 的规则。

11.2 完成 正常 复核与失败

表 10 四种容易混淆的状态

状态或指标	判断来源	正确解读
特征生成完成	NPZ 和元数据产生并可汇总	该样本具有完整输出结构
ok	未触发当前告警规则	未发现配置所定义的质量提示
review_required	存在低分、数字读法或视觉等提示	数据保留，但需要进一步回查
failed	样本处理出现未能完成的异常	不能视为成功交付，需要处理错误
核验错误列表为空	自动结构与聚合复算检查	通过实现一致性检查，不是人工精度证明

现有结果记录 100 条完成、1931 个词位置、1601 个视觉有效词、342 个低分词和 69 条需要复核的样本。词级时间与语音覆盖均为 100%，视觉覆盖为 82.91%。这些数字能说明产生了什么、可用性如何；由于没有人工逐词边界真值，不能据此计算或宣称对齐准确率。

11.3 重聚合和打包的边界

reaggregate.py 先检查原特征文件哈希，再读取保存的原始帧数组重算两类词级统计量。它要求贡献帧映射和掩码保持不变，因此当前脚本适合只调整数值累加精度，不是任意更改词边界或重叠规则的通用迁移工具。完成后登记旧、新文件哈希和聚合代码版本；原始深度特征、时间与掩码不变。

package_results.py 读取 validation.json，要求产出 100 条且错误列表为空，再压缩代码、配置、结果和模型清单。包内不放原视频、虚拟环境和预训练大权重，最后检查压缩完整性、NPZ 数量和 50,000,000 字节大小门限。打包依赖最近的验证记录，因此输出改动后应先重新汇总，而非直接使用旧报告打包。

12 架构局限 维护重点与阅读路线

12.1 断点续跑已经检查什么

当前缓存跳过条件包括：元数据与特征文件存在、处理指纹一致、源视频哈希一致、输出特征哈希一致。处理指纹由配置、`pipeline.py`、`alignment.py` 及模型清单共同形成。它能识别这些代码或配置变化，以及源视频与数值文件损坏，避免单纯依据文件名跳过计算。

但该条件没有直接逐条比较当前转写与旧元数据，也没有把标签表内容哈希纳入处理指纹。若只修改表格文本而不改视频，缓存可能被沿用；随后 `summarize` 的原文一致性检查能够报告不一致。环境版本虽被记录，也没有整体进入缓存指纹；运行时的模型清单摘要检查不等于每次都重新计算全部模型权重哈希。这些属于实际缓存边界，不应写成“任何输入变化都会自动重算”。

12.2 现有实现的维护重点

表 11 维护时应关注的限制及改进方向

方面	当前边界	可考虑的改进
配置绑定	模型名称字符串不控制本地权重切换；部分阈值在代码中	统一模型加载契约和参数来源
缓存完整性	文本、依赖版本未完整进入缓存判据	纳入逐样本文本与环境签名
结果写入	特征与元数据分两步写入	先写临时文件再一致提交
全量失败管理	部分读取和缓存校验在逐样本异常处理之外	细化异常边界并保证资源释放
审计记录	运行环境快照覆盖写入，日志持续追加	引入独立运行编号与历史目录
视觉与时间可靠性	无说话人确认及人工边界真值	增加回看标注与定量验证
后续任务接口	768/50/110 维不同于附件 2	保持各任务约定，不直接替换

这些建议是对现有代码的架构分析，不表示相关改进已经实现。现有结构适合 100 条短视频的离线提取与论文复现；扩展到更长视频或大量样本时，还需评估声学模型内存、CTC 回溯表大小、逐词遍历源帧的计算量，以及并发时模型资源和输出文件的管理。

12.3 如何带着问题阅读项目

想理解整体执行，先读 `run.ps1` 与 `pipeline` 的 `main`；想理解一个样本如何形成，定位 `process_one`；想看模型作用，分别读 `Extractor` 的文本、对齐、声学与视觉四个方法；想核对时间关系，读 `alignment.py`；想理解输出，读 `read_features.py` 与一对 NPZ、JSON；想评价是否可信，检查 `summarize.py`、`validation.json` 及人工复核记录。

资料依据

项目依据：question1 目录下现有提取、对齐、读取、准备、核验、重聚合和打包文件；本地两类模型 `config.json`；`results` 中的运行环境、验证和全量汇总记录。

模型资料：DistilBERT 模型卡 <https://huggingface.co/distilbert/distilbert-base-uncased>；Wav2Vec2 模型卡 <https://huggingface.co/facebook/wav2vec2-base-960h>。

工具资料：openSMILE <https://audeering.github.io/opensmile-python/>；MediaPipe Face Landmarker https://developers.google.com/edge/mediapipe/solutions/vision/face_landmarker。公开资料用于说明工具定位，结构参数以本地保存的配置为准。